

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 3. Python Data Types

Lecture: 24. Base Conversion

Section 1: Lecture Summary

This lecture focuses on **base conversion functions in Python**, continuing from previous discussions on number systems. It reviews common number systems — decimal (base 10), binary (base 2), octal (base 8), and hexadecimal (base 16) — and relates them to computer systems. The core topic is how Python provides built-in functions to convert decimal integers into binary, octal, and hexadecimal string representations. The instructor explains that these functions take integer inputs and return strings with specific prefixes indicating the base. Practical examples using Python's ``bin()``, ``oct()``, and ``hex()`` functions are demonstrated in a Jupyter Notebook environment. The lecture clarifies acceptable input types and highlights that these functions only accept integers (including boolean values treated as integers), but not floats. Overall, students learn the purpose, usage, and output format of these base conversion functions.

Section 2: Key Concepts and Explanations

- Number Systems Overview:

- **Decimal (Base 10):** Uses digits 0 to 9.
- **Binary (Base 2):** Uses digits 0 and 1.
- **Octal (Base 8):** Uses digits 0 to 7.
- **Hexadecimal (Base 16):** Uses digits 0 to 9 and letters A to F.

- Base Conversion Functions in Python:

- These functions convert a decimal integer input into a string representation of a number in a different base.

Function	Converts decimal to	Input Type	Output Type	Prefix in Output String
<code>`bin()`</code>	Binary (base 2)	Integer	String	<code>`0b`</code>
<code>`oct()`</code>	Octal (base 8)	Integer	String	<code>`0o`</code>
<code>`hex()`</code>	Hexadecimal (base 16)	Integer	String	<code>`0x`</code>

- **Characteristics:**

- All functions accept only integer inputs; passing a float raises a ``TypeError``.
- Boolean values (``True`` and ``False``) are accepted because Python treats ``True`` as integer ``1`` and ``False`` as ``0``.
- Output of these functions is always a string including a prefix. For example, ``bin(10)`` returns the string ``"0b1010"``.

- **Additional Notes:**

- To convert strings representing numbers in other bases to decimal integers, Python uses the ``int()`` function with a base parameter (not covered in detail here).
- The prefixes (``0b``, ``0o``, ``0x``) are part of Python's notation for literals of binary, octal, and hexadecimal numbers.
- These functions are ready-made or **built-in functions** — no need to define them manually.

Section 3: Example Code and Use Cases

Example 1: Convert decimal 10 to binary, octal, and hexadecimal

Output: 0b1010 0o12 0xa

```
a = 10
s1 = bin(a)
s2 = oct(a)
s3 = hex(a)

print(s1, s2, s3)
```

Explanation: Converts decimal 10 to binary (`0b1010`), octal (`0o12`), and hexadecimal (`0xa`). Output is string with prefixes.

Example 2: Convert decimal 15 to binary, octal, and hexadecimal

```
s1 = bin(15)
s2 = oct(15)
s3 = hex(15)

print(s1) # Output: 0b1111
print(s2) # Output: 0o17
print(s3) # Output: 0xf
```

Explanation: Converts decimal 15 showing binary (`0b1111`), octal (`0o17`), and hexadecimal (`0xf`).

Example 3: Using boolean values as input

```
s1 = bin(True)
print(s1) # Output: 0b1
```

Explanation: Boolean `True` treated as integer 1, converted to binary string.

Example 4: Passing a float value to bin()

Raises TypeError: 'float' object cannot be interpreted as an integer

```
s1 = bin(1.0)
```

Explanation: Passing a float raises an error since only integers are accepted.

Combined example with variable assignment

```
a = 10
x1 = bin(a)    # Binary string of 10
x2 = oct(a)    # Octal string of 10
x3 = hex(a)    # Hexadecimal string of 10

print(x1, x2, x3)
```

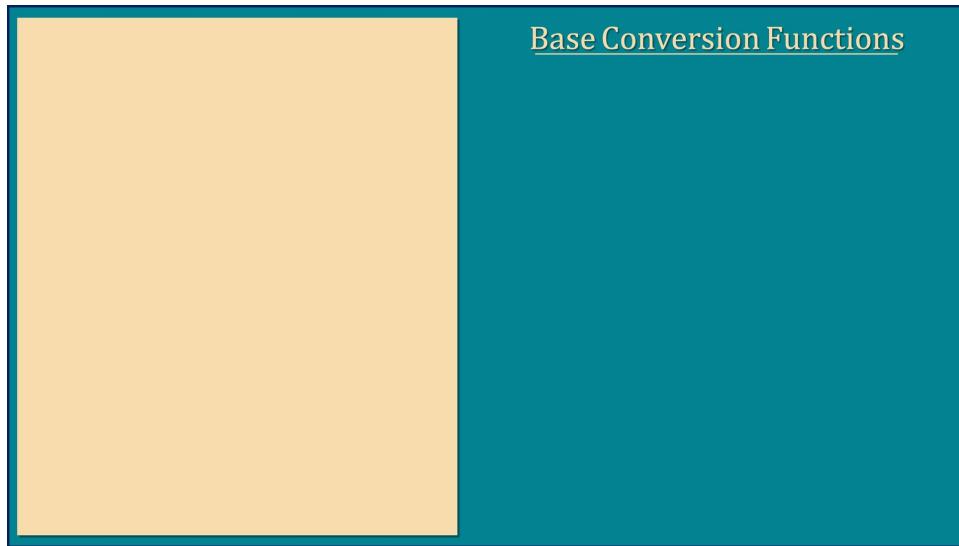
Explanation: Assigns conversion results to variables and prints them, useful in applications where base representation is needed.

Section 4: Key Takeaways

- Python provides built-in functions `bin()`, `oct()`, and `hex()` to convert decimal integers to binary, octal, and hexadecimal string representations.
- These functions **accept only integer inputs**; passing floats results in errors.
- Boolean values are accepted as integers (`True`=1, `False`=0).
- Output strings from these functions include base-specific prefixes: `'0b'`, `'0o'`, `'0x'`.
- Base conversion functions always **return strings**, not integers.
- Understanding number systems and how Python represents them is crucial for programming tasks involving different numeral bases.
- These built-in functions provide easy and reliable ways to convert number bases without manual computation.
- Practice using these conversions in Python environments like Jupyter Notebook to become comfortable with their syntax and output formats.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Base Conversion Functions

Number Systems

Base Conversion Functions

Number Systems

Decimal

Base Conversion Functions

Number Systems

Decimal **base 10**
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Base Conversion Functions

Number Systems

Decimal **base 10**
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Binary

Base Conversion Functions

Number Systems

Decimal	base 10 {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Binary	base 2 {0, 1}

Base Conversion Functions

Number Systems

Decimal	base 10 {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Binary	base 2 {0, 1}
Octal	

Base Conversion Functions

Number Systems

Decimal	base 10 {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Binary	base 2 {0, 1}
Octal	base 8 {0, 1, 2, 3, 4, 5, 6, 7}

Base Conversion Functions

Number Systems

Decimal	base 10 {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Binary	base 2 {0, 1}
Octal	base 8 {0, 1, 2, 3, 4, 5, 6, 7}
Hexadecimal	

Base Conversion Functions

Number Systems

Decimal **base 10**
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Binary **base 2**
{0, 1}

Octal **base 8**
{0, 1, 2, 3, 4, 5, 6, 7}

Hexadecimal **base 16**
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}

Base Conversion Functions

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`bin(int)` – Binary

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`bin(int)` – Binary

`oct(int)` – Octal

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`bin(int)` – Binary

`oct(int)` – Octal

`hex(int)` – Hexadecimal

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`bin( int )` – Binary

`oct( int )` – Octal

`hex( int )` – Hexadecimal

`int( str )` – Decimal

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`string bin( int )` – Binary

`string oct( int )` – Octal

`string hex( int )` – Hexadecimal

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`string bin( int )` – Binary

`string oct( int )` – Octal

`string hex( int )` – Hexadecimal

result

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Built-in Functions

`string bin( int )` – Binary

`string oct( int )` – Octal

`string hex( int )` – Hexadecimal

result – String

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

string bin(int)

string oct(int)

string hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

bin(int)

oct(int)

hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

10

bin(int)

oct(int)

hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

10
↓
bin()
oct(int)
hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

bin(10)
oct(int)
hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

bin(10)

oct(int)

hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

'0b1010' ← bin(10)

oct(int)

hex(int)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`oct( int ) 10`

`hex( int )`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`oct( ) 10`

`hex( int )`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`oct( 10 )`

`hex( int )`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`'0o12' ← oct( 10 )`

`hex( int )`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`'0o12' ← oct( 10 )`

`hex( int ) 10`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`'0o12' ← oct( 10 )`

`hex(   ) 10`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

`'0b1010' ← bin( 10 )`

`'0o12' ← oct( 10 )`

`hex(10 )`

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Binary

s1 = bin(a)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Binary

s1 = bin(a)



Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Binary

s1 = bin(a)

s1
'0b1010'

Octal

s2 = oct(a)

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Binary

s1 = bin(a)

s1
'0b1010'

Octal

s2 = oct(a)

s2
'0o12'

Deci	Bin	Octal	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Base Conversion Functions

Decimal Number

a = 10

Binary **s1 = bin(a)** s1 → '0b1010'

Octal **s2 = oct(a)** s2 → '0o12'

Hexa **s3 = hex(a)** s3 → '0xA'

Base Conversion Functions

s1 = bin(10)

s2 = oct(10)

s3 = hex(10)

Base Conversion Functions

```
s1 = bin( 10 )
```

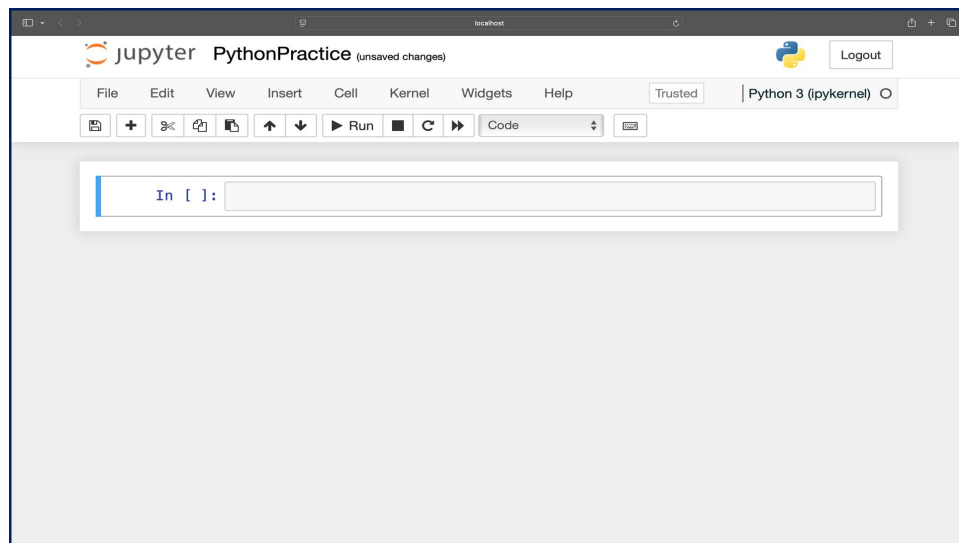
```
s2 = oct( 10 )
```

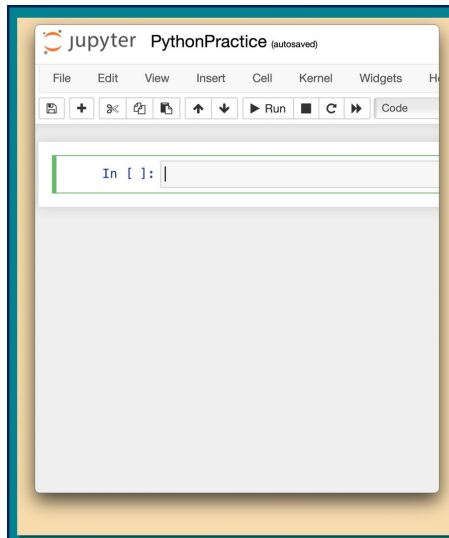
```
s3 = hex( 10 )
```

```
s1 = bin( 15 )
```

```
s2 = oct( 15 )
```

```
s3 = hex( 15 )
```





Base Conversion Functions

s1 = bin(10)

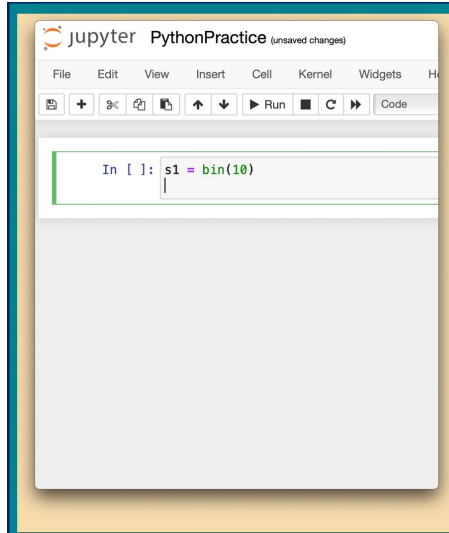
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)

A screenshot of the Jupyter PythonPractice interface. The title bar shows 'jupyter PythonPractice (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for adding, saving, undo, redo, running, and other actions. A code cell is active, showing the code 'In []: s1 = bin(10)'.

Base Conversion Functions

s1 = bin(10)

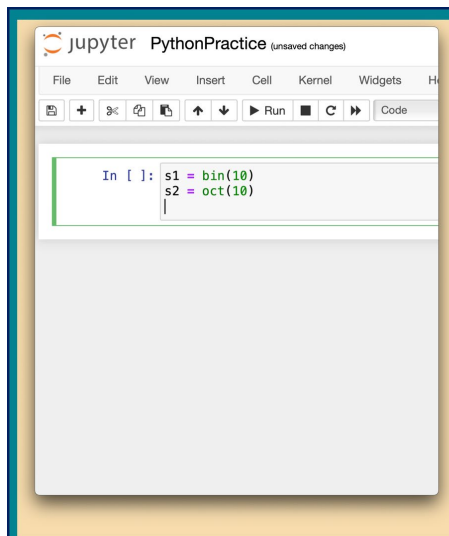
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)

A screenshot of the Jupyter PythonPractice interface, similar to the one above. The code cell now contains two lines of code: 'In []: s1 = bin(10)' and 's2 = oct(10)'.

Base Conversion Functions

s1 = bin(10)

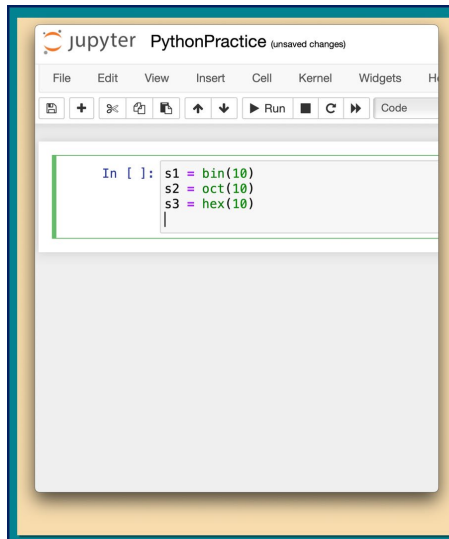
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and output viewing. A code cell is active, containing the following Python code:

```
In [ ]: s1 = bin(10)
        s2 = oct(10)
        s3 = hex(10)
        |
```

Base Conversion Functions

s1 = bin(10)

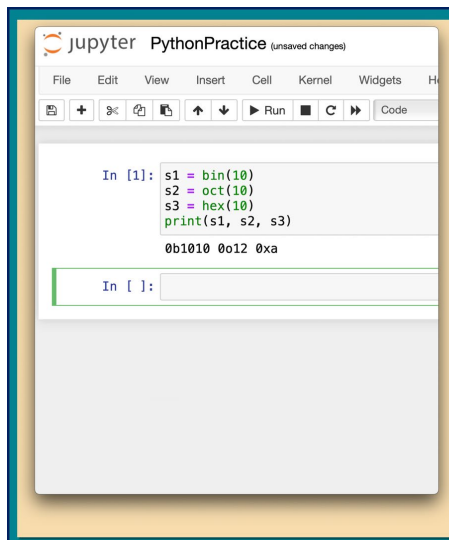
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface is similar to the first image. A code cell is active, containing the following Python code:

```
In [1]: s1 = bin(10)
        s2 = oct(10)
        s3 = hex(10)
        print(s1, s2, s3)
```

Below the code cell, the output of the execution is displayed:

```
0b1010 0o12 0xa
```

Below the output, there is an empty code cell:

```
In [ ]:
```

Base Conversion Functions

s1 = bin(10)

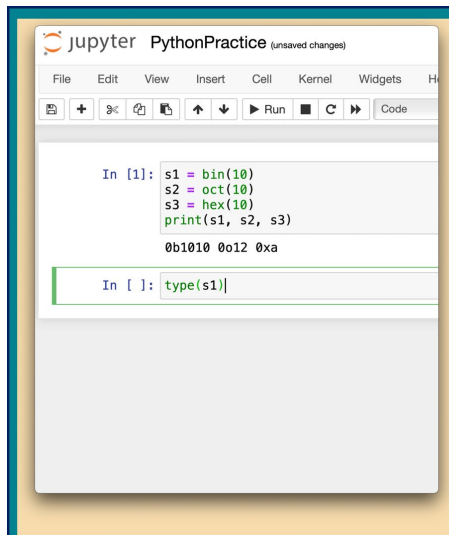
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". Below the menu is a toolbar with icons for saving, undo, redo, and running code. The first code cell, labeled "In [1]:", contains the following Python code:

```
s1 = bin(10)
s2 = oct(10)
s3 = hex(10)
print(s1, s2, s3)
```

 The output of this cell is "0b1010 0o12 0xa". The second code cell, labeled "In []:", contains the code

```
type(s1)
```

 and is currently empty.

Base Conversion Functions

s1 = bin(10)

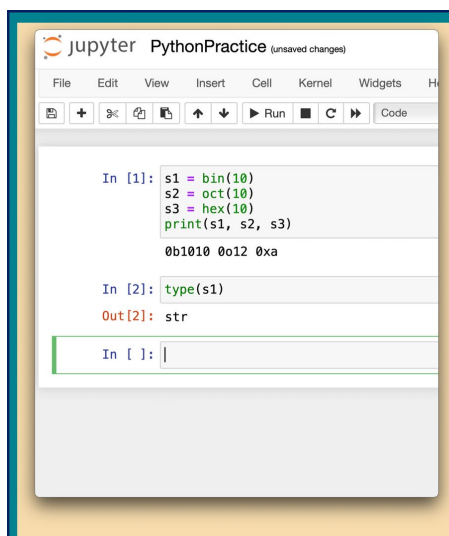
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". Below the menu is a toolbar with icons for saving, undo, redo, and running code. The first code cell, labeled "In [1]:", contains the following Python code:

```
s1 = bin(10)
s2 = oct(10)
s3 = hex(10)
print(s1, s2, s3)
```

 The output of this cell is "0b1010 0o12 0xa". The second code cell, labeled "In [2]:", contains the code

```
type(s1)
```

 and the output is "Out [2]: str". The third code cell, labeled "In []:", is currently empty.

Base Conversion Functions

s1 = bin(10)

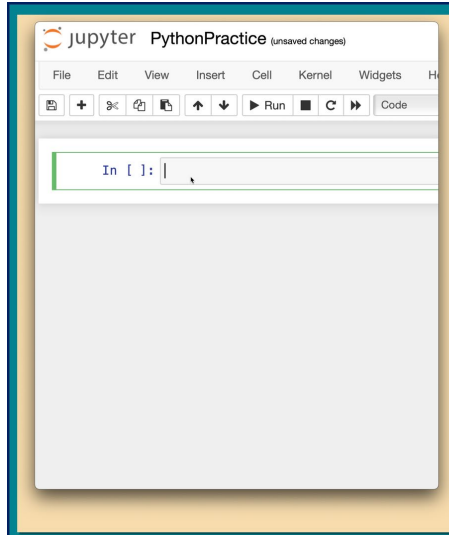
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



Base Conversion Functions

s1 = bin(10)

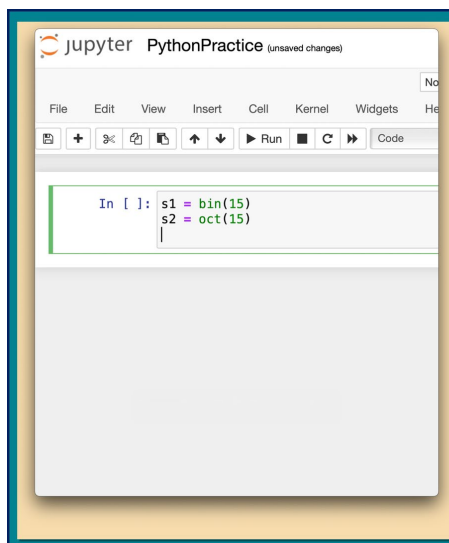
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



Base Conversion Functions

s1 = bin(10)

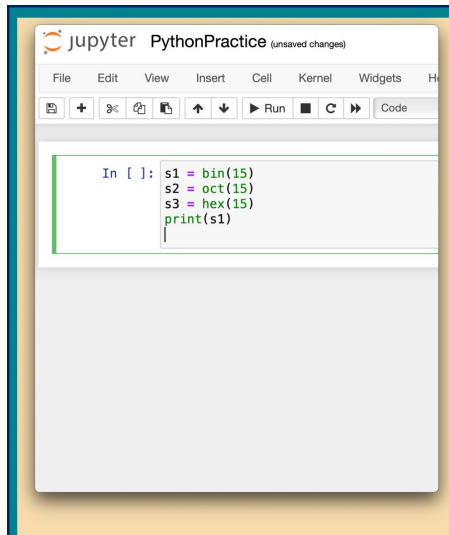
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and navigation. The code cell contains the following Python code:

```
In [ ]: s1 = bin(15)
        s2 = oct(15)
        s3 = hex(15)
        print(s1)
```

Base Conversion Functions

s1 = bin(10)

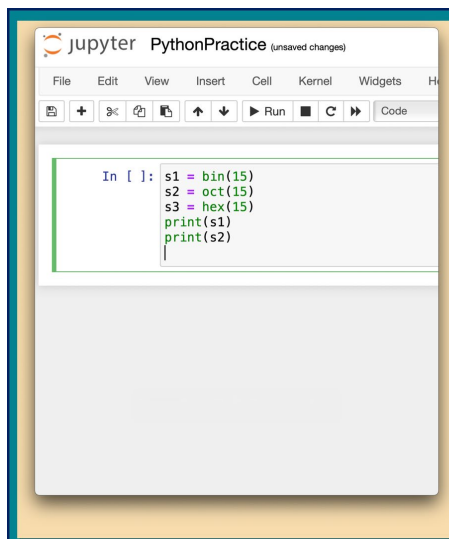
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and navigation. The code cell contains the following Python code:

```
In [ ]: s1 = bin(15)
        s2 = oct(15)
        s3 = hex(15)
        print(s1)
        print(s2)
```

Base Conversion Functions

s1 = bin(10)

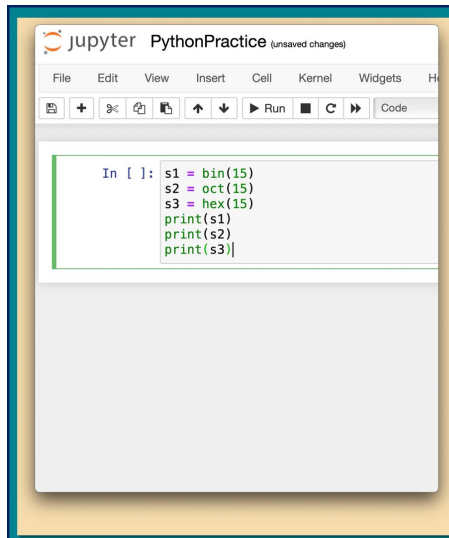
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



The image shows a Jupyter Notebook window titled "jupyter PythonPractice (unsaved changes)". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". Below the menu is a toolbar with icons for file operations, cell navigation, and execution. The main area contains a code cell with the following Python code:

```
In [ ]: s1 = bin(15)
        s2 = oct(15)
        s3 = hex(15)
        print(s1)
        print(s2)
        print(s3)
```

Base Conversion Functions

s1 = bin(10)

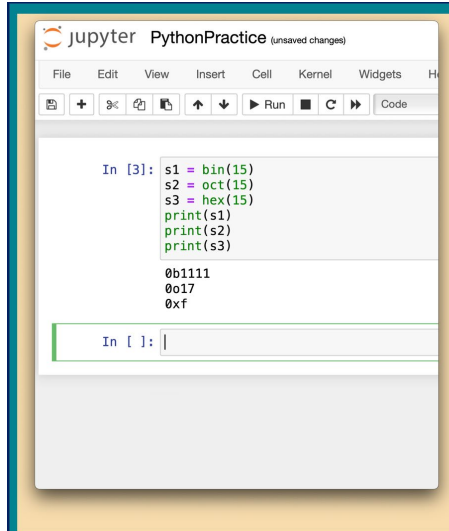
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)

A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". Below the menu is a toolbar with icons for adding, deleting, and running code cells. The main area shows a code cell with the following Python code:

```
In [3]: s1 = bin(15)
        s2 = oct(15)
        s3 = hex(15)
        print(s1)
        print(s2)
        print(s3)
```

The output of the code is displayed below the code cell:

```
0b1111
0o17
0xf
```

Below the output, there is an empty code cell with the prompt "In []: |".

Base Conversion Functions

s1 = bin(10)

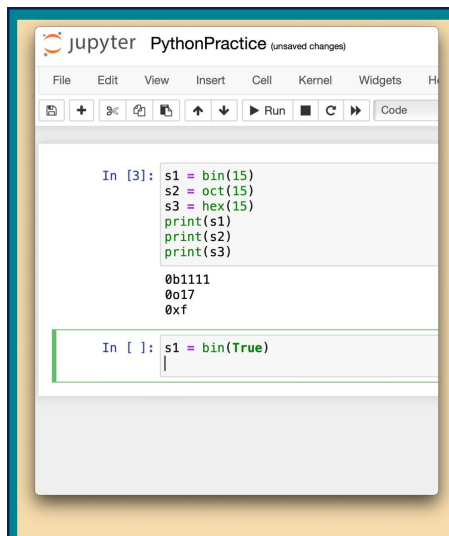
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)

A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". Below the menu is a toolbar with icons for adding, deleting, and running code cells. The main area shows a code cell with the following Python code:

```
In [3]: s1 = bin(15)
        s2 = oct(15)
        s3 = hex(15)
        print(s1)
        print(s2)
        print(s3)
```

The output of the code is displayed below the code cell:

```
0b1111
0o17
0xf
```

Below the output, there is a new code cell with the following Python code:

```
In [ ]: s1 = bin(True)
        |
```

Base Conversion Functions

s1 = bin(10)

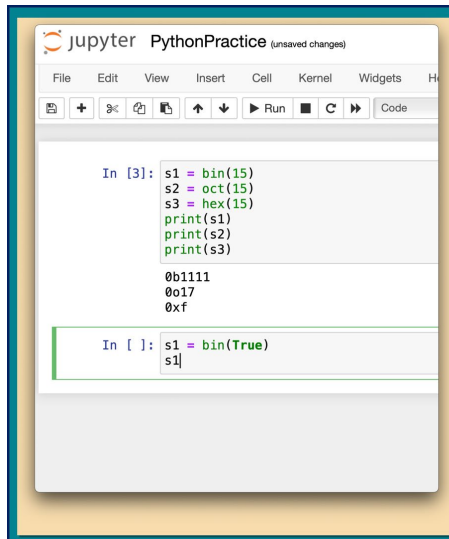
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and navigation. The notebook contains two input cells. The first cell, labeled "In [3]:", contains the following code: `s1 = bin(15)`, `s2 = oct(15)`, `s3 = hex(15)`, `print(s1)`, `print(s2)`, and `print(s3)`. The output of this cell is displayed below the code: `0b1111`, `0o17`, and `0xf`. The second cell, labeled "In []:", contains the code `s1 = bin(True)` and `s1`. The output of this cell is not yet displayed.

Base Conversion Functions

s1 = bin(10)

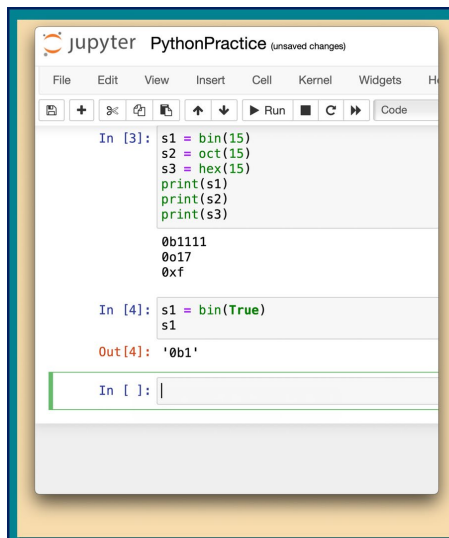
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A screenshot of a Jupyter Notebook titled "PythonPractice (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and navigation. The notebook contains two input cells. The first cell, labeled "In [3]:", contains the following code: `s1 = bin(15)`, `s2 = oct(15)`, `s3 = hex(15)`, `print(s1)`, `print(s2)`, and `print(s3)`. The output of this cell is displayed below the code: `0b1111`, `0o17`, and `0xf`. The second cell, labeled "In [4]:", contains the code `s1 = bin(True)` and `s1`. The output of this cell is displayed below the code: `Out[4]: '0b1'`. The third cell, labeled "In []:", is currently empty.

Base Conversion Functions

s1 = bin(10)

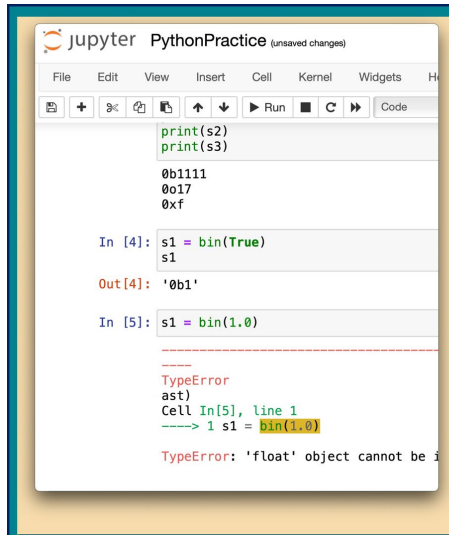
s2 = oct(10)

s3 = hex(10)

s1 = bin(15)

s2 = oct(15)

s3 = hex(15)



A Jupyter Notebook interface titled "PythonPractice (unsaved changes)". The code cell contains the following Python code:

```
print(s2)
print(s3)

0b1111
0o17
0xf

In [4]: s1 = bin(True)
s1
Out[4]: '0b1'

In [5]: s1 = bin(1.0)
```

The output of the first cell shows the binary, octal, and hexadecimal representations of the integer 15. The second cell shows the binary representation of the boolean value True. The third cell attempts to convert the float 1.0 to a binary string, which results in a `TypeError: 'float' object cannot be interpreted as an integer`.

Base Conversion Functions

`s1 = bin( 10 )`

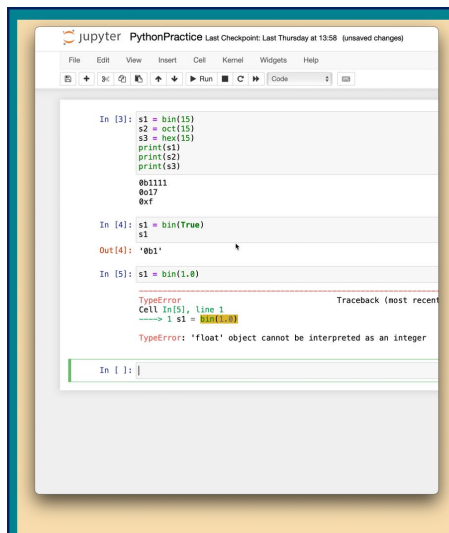
`s2 = oct( 10 )`

`s3 = hex( 10 )`

`s1 = bin( 15 )`

`s2 = oct( 15 )`

`s3 = hex( 15 )`



A Jupyter Notebook interface titled "PythonPractice Last Checkpoint: Last Thursday at 13:58 (unsaved changes)". The code cell contains the following Python code:

```
In [3]: s1 = bin(15)
s2 = oct(15)
s3 = hex(15)
print(s1)
print(s2)
print(s3)

0b1111
0o17
0xf

In [4]: s1 = bin(True)
s1
Out[4]: '0b1'

In [5]: s1 = bin(1.0)
```

The output of the first cell shows the binary, octal, and hexadecimal representations of the integer 15. The second cell shows the binary representation of the boolean value True. The third cell attempts to convert the float 1.0 to a binary string, which results in a `TypeError: 'float' object cannot be interpreted as an integer`.

Base Conversion Functions

`s1 = bin( 10 )`

`s2 = oct( 10 )`

`s3 = hex( 10 )`

`s1 = bin( 15 )`

`s2 = oct( 15 )`

`s3 = hex( 15 )`

